

DATABASE MANAGEMENT LAB

LAB Report

Submitted in partial fulfilment of the requirement of the degree of

BACHELORS OF TECHNOLOGY

to

K.R Mangalam University



Submitted by:

Nakul Arora

(2401010025)

Submitted to

Ms. Mansi Kajal

Department of Computer Science and Engineering

School of Engineering and Technology

K.R Mangalam University, Gurugram- 122001, India

SESSION – 2025-2026

Experiment : 1

Question 1. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COLLEGE DATABASE

STUDENT (USN, SName, Address, Phone, Gender)

SEMSEC (SSID, Sem, Sec)

CLASS (USN, SSID)

SUBJECT (Subcode, Title, Sem, Credits)

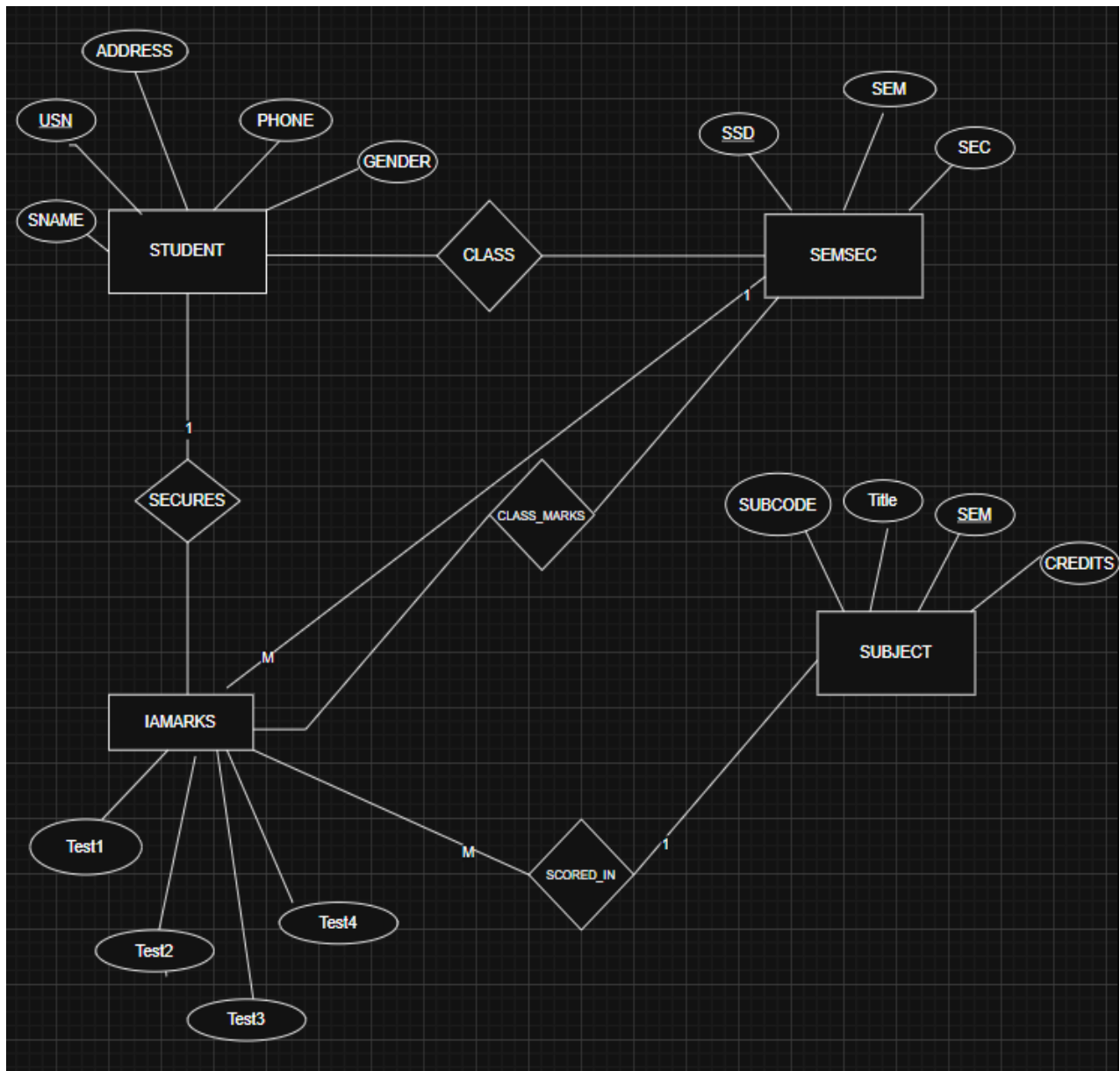
IAMARKS (USN, Subcode, SSID, Test1, Test2, Test3, FinalIA)

Solution:

1. The entities **STUDENT**, **SEMSEC**, **SUBJECT**, and **IAMARKS** are identified and represented using rectangles.
2. The attributes are represented using ellipses:
 - o **STUDENT**: USN (Primary Key), SName, Address, Phone, Gender
 - ♣ USN is underlined as it is the Primary Key.
 - ♣ Phone is shown as a single-valued attribute.
 - o **SEMSEC**: SSID (Primary Key), Sem, Sec
 - ♣ SSID is underlined as the Primary Key.
 - o **SUBJECT**: Subcode (Primary Key), Title, Sem, Credits
 - ♣ Subcode is underlined as the Primary Key.
 - o **IAMARKS**: USN, Subcode, SSID, Test1, Test2, Test3, FinalIA
 - ♣ USN, Subcode, and SSID together form a composite key.
 - ♣ Test1, Test2, Test3, and FinalIA are normal attributes.
3. Primary keys are represented by underlining the attribute names.
4. **CLASS** is drawn as a relationship (diamond shape) between **STUDENT** and **SEMSEC**.
5. The relationship between **STUDENT** and **SEMSEC** through **CLASS** is **M : 1**, meaning many students belong to one semester and section.
6. The **M : N relationship between STUDENT and SUBJECT** is resolved using the associative entity **IAMARKS**.
7. The relationship **STUDENT – IAMARKS** is **1 : M**, indicating one student can have multiple internal assessment records.
8. The relationship **SUBJECT – IAMARKS** is **1 : M**, indicating one subject can have multiple IA records for different students.

9. The relationship **SEMSEC – IAMARKS** is **1 : M**, showing that marks are maintained semester-section wise.
10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



Experiment : 2

Question 2. Consider following databases and draw ER diagram and convert entities and relationships to relation table for a given scenario:

COMPANY DATABASE

EMPLOYEE (SSN, Name, Address, Salary, Sex, SuperSSN, DNO)

DEPARTMENT (DNo, DName, MgrSSN, MgrStartDate)

PROJECT (PNo, PName, PLocation, DNO)

DLOCATION (DNo, DLoc)

WORKS_ON (SSN, PNo, Hours)

MANAGES (SSN, DNO)

Solution:

1. The entities **EMPLOYEE, DEPARTMENT, PROJECT, and DLOCATION** are identified and represented using rectangles.
2. The attributes are represented using ellipses:

o **EMPLOYEE:** SSN (Primary Key), Name, Address, Salary, Sex, SuperSSN, DNO

♣ SSN is underlined as it is the Primary Key.

♣ SuperSSN is a foreign key referencing EMPLOYEE (self-relationship).

♣ DNO is a foreign key referencing DEPARTMENT.

o **DEPARTMENT:** DNo (Primary Key), DName, MgrSSN, MgrStartDate

♣ DNo is underlined as the Primary Key.

♣ MgrSSN is a foreign key referencing EMPLOYEE.

o **PROJECT:** PNo (Primary Key), PName, PLocation, DNO

♣ PNo is underlined as the Primary Key.

♣ DNO is a foreign key referencing DEPARTMENT.

o **DLOCATION:** DNo, DLoc

♣ DNo and DLoc together form a composite key.

♣ DNo is a foreign key referencing DEPARTMENT.

o **WORKS_ON:** SSN, PNo, Hours

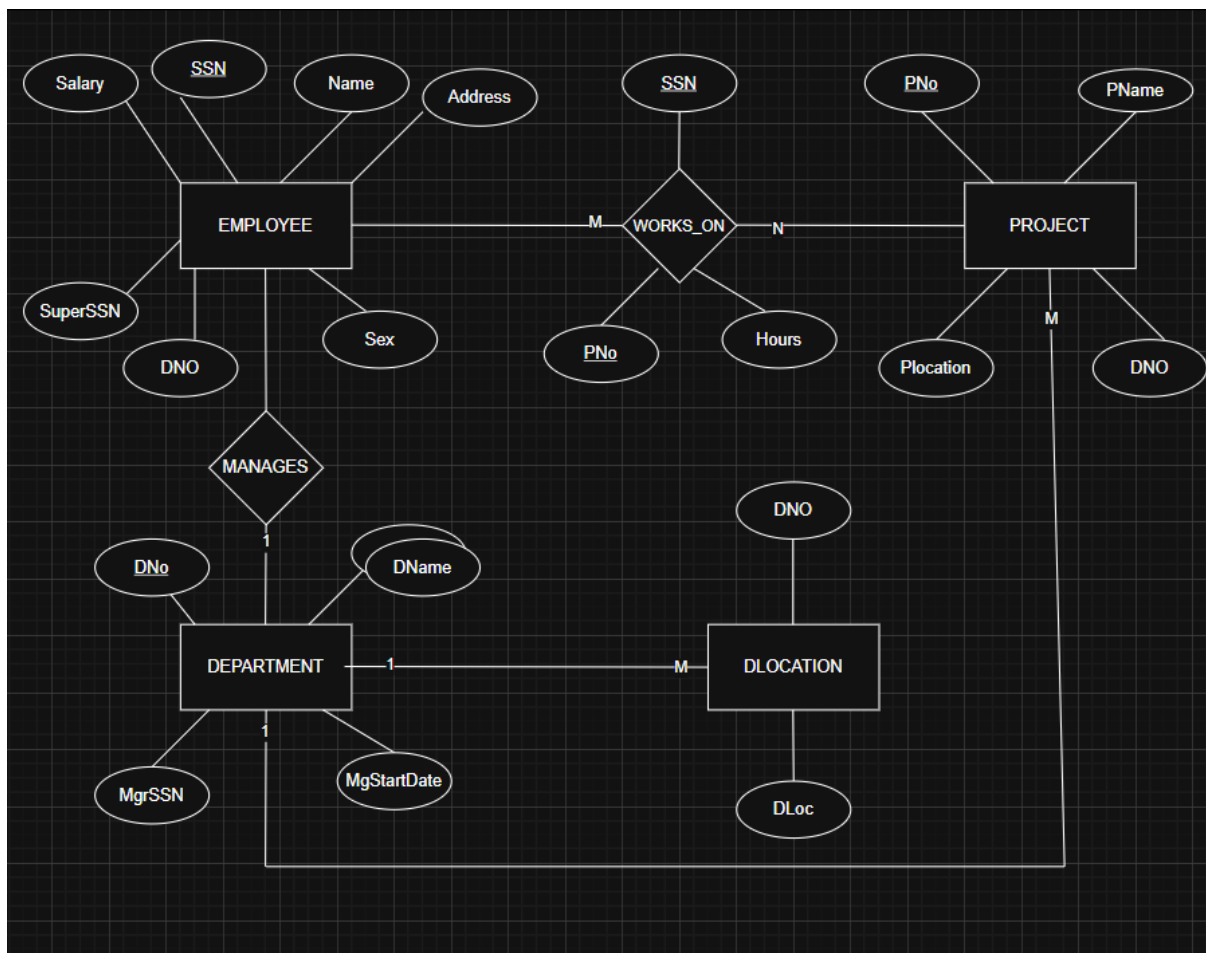
♣ SSN and PNo together form a composite key.

♣ Hours is a descriptive attribute of the relationship.

3. Primary keys are represented by underlining the attribute names.
4. The relationship **WORKS_ON** is drawn as a diamond between **EMPLOYEE** and **PROJECT** with **M : N cardinality**, indicating that:
 - o One employee can work on many projects.
 - o One project can have many employees.

- Hours attribute represents the number of hours worked.
- 5. The relationship **MANAGES** is drawn between **EMPLOYEE** and **DEPARTMENT** with **1 : 1 cardinality**, indicating:
 - One employee manages one department.
 - Each department has one manager.
- 6. The relationship between **EMPLOYEE** and **DEPARTMENT** (via DNO) is **M : 1**, meaning:
 - Many employees work in one department.
- 7. The relationship between **DEPARTMENT** and **PROJECT** is **1 : M**, meaning:
 - One department controls many projects.
 - Each project belongs to one department.
- 8. The relationship between **DEPARTMENT** and **DLOCATION** is **1 : M**, meaning:
 - One department can have multiple locations.
- 9. A recursive relationship exists in **EMPLOYEE** through **SuperSSN**, representing that:
 - One employee can supervise many employees.
- 10. All entities, attributes, relationships, primary keys, foreign keys, and cardinalities are clearly represented in the ER diagram.

ER DIAGRAM:



Experiment : 3

Question : Database Schema for a customer-sale scenario

Customer(Cust id : integer, cust_name: string)

Item(item_id: integer, item_name: string, price: integer)

Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

Output:

```
1 • CREATE DATABASE CUSTOMER_SALE;
2 • USE CUSTOMER_SALE;
3 • CREATE TABLE CUSTOMER(
4   CUST_ID INT PRIMARY KEY,
5   CUST_NAME VARCHAR(100)
6 );
7 • CREATE TABLE ITEM (
8   ITEM_ID INT PRIMARY KEY,
9   ITEM_NAME VARCHAR(100),
10  PRICE INT);
11 • CREATE TABLE SALE (
12  BILL_NO INT ,
13  BILL_DATA DATE,
14  CUST_ID INT,
15  ITEM_ID INT,
16  QTY_SOLD INT ,
17  FOREIGN KEY (CUST_ID) REFERENCES CUSTOMER(CUST_ID),
18  FOREIGN KEY (ITEM_ID) REFERENCES ITEM(ITEM_ID)
19 );
```

2. Insert around 10 records in each of the tables.

```
20 • INSERT INTO
21  CUSTOMER(CUST_ID ,CUST_NAME)
22  VALUES
23  (1,'nakul'),
24  (2,'Harsh'),
25  (3,'Ankit'),
26  (4,'Aryan'),
27  (5,'Hridyanshu'),
28  (6,'Ashok'),
29  (7,'Alok'),
30  (8,'Aman'),
31  (9,'Ashish'),
32  (10,'Mohit');
```



	CUST_ID	CUST_NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	Hridyanshu
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
*	NULL	NULL

```

INSERT INTO ITEM VALUES
(101, 'Pen', 50),
(102, 'Notebook', 120),
(103, 'Bag', 800),
(104, 'Bottle', 250),
(105, 'Shoes', 1500),
(106, 'Watch', 2200),
(107, 'Mouse', 500),
(108, 'Keyboard', 700),
(109, 'Charger', 300),
(110, 'Headphones', 1800);
SELECT * FROM ITEM;

```



	ITEM_ID	ITEM_NAME	PRICE
▶	101	Pen	50
	102	Notebook	120
	103	Bag	800
	104	Bottle	250
	105	Shoes	1500
	106	Watch	2200
	107	Mouse	500
	108	Keyboard	700
	109	Charger	300
	110	Headphones	1800
•	NULL	NULL	NULL

```

49 • INSERT INTO SALE VALUES
50 (1,CURDATE(),1,103,1),
51 (2,CURDATE(),2,101,3),
52 (3,CURDATE(),2,105,2),
53 (4,'2026-02-15',3,104,1),
54 (5,'2026-02-14',4,106,1),
55 (6,'2026-02-13',5,102,4),
56 (7,CURDATE(),6,110,1),
57 (8,'2026-02-16',7,108,2),
58 (9,CURDATE(),5,109,5),
59 (10,'2026-02-12',8,107,1);
60 • SELECT * FROM SALE;

```



	BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	QTY_SOLD
▶	1	2026-02-23	1	103	1
	2	2026-02-23	2	101	3
	3	2026-02-23	2	105	2
	4	2026-02-15	3	104	1
	5	2026-02-14	4	106	1
	6	2026-02-13	5	102	4
	7	2026-02-23	6	110	1
	8	2026-02-16	7	108	2
	9	2026-02-23	5	109	5
	10	2026-02-12	8	107	1

4. List all the bills for the current date with the customer names and item numbers

```

61
62 • SELECT S.BILL_NO, C.CUST_NAME, S.ITEM_ID
63 FROM SALE S
64 JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
65 WHERE S.BILL_DATA = CURDATE();
66

```



	BILL_NO	CUST_NAME	ITEM_ID
▶	1	Nakul	103
	2	Harsh	101
	3	Harsh	105
	3	Harsh	105
	7	Ashok	110
	9	Aman	110

5. List the total Bill details with the quantity sold, price of the item and the final amount.

```

67 • SELECT
68     S.BILL_NO,
69     C.CUST_NAME,
70     I.ITEM_NAME,
71     S.QTY_SOLD,
72     I.PRICE,
73     (S.QTY_SOLD * I.PRICE) AS FINAL_AMOUNT
74 FROM SALE S
75 JOIN CUSTOMER C ON S.CUST_ID = C.CUST_ID
76 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
--

```

	BILL_NO	CUST_NAME	ITEM_NAME	QTY_SOLD	PRICE	FINAL_AMC
▶	1	Nakul	Bag	1	800	800
	2	Harsh	Pen	3	101	150
	3	Harsh	Shoes	2	1500	3000
	4	Ankit	Bottle	1	250	250
	5	Aryan	Watch	1	2200	2200
	6	Hridyanshu	Notebook	4	120	480
	7	Ashok	Headphones	1	1800	1800
	8	Alok	Keyboard	2	700	1400
	9	Hridyanshu	Charger	5	300	1500
	10	Aman	Mouse	1	500	500

6. List the details of the customer who have bought a product which has a price>200

```

79 • SELECT DISTINCT C.*
80 FROM CUSTOMER C
81 JOIN SALE S ON C.CUST_ID = S.CUST_ID
82 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
83 WHERE I.PRICE > 200;

```

	CUST_ID	CUST_NAME
▶	1	Nakul
	3	Ankit
	2	Harsh
	4	Aryan
	8	Aman
	7	Alok
	5	Hridyanshu
	6	Ashok

7. Give a count of how many products have been bought by each customer

```

86 • SELECT C.CUST_NAME, SUM(S.QTY_SOLD) AS TOTAL_PRODUCTS_BOUGHT
87 FROM CUSTOMER C
88 JOIN SALE S ON C.CUST_ID = S.CUST_ID
89 GROUP BY C.CUST_NAME;
90

```

	CUST_NAME	TOTAL_PRODUCTS_BOUGHT
▶	Nakul	1
	Harsh	5
	Ankit	1
	Aryan	1
	Hridyanshu	9
	Ashok	1
	Alok	2
	Aman	1

8. Give a list of products bought by a customer having cust_id as 5.

```

92 • SELECT I.ITEM_NAME, S.QTY_SOLD
93 FROM SALE S
94 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
95 WHERE S.CUST_ID = 5;
96

```

	ITEM_NAME	QTY_SOLD
▶	Notebook	4
	Charger	5

9. List the item details which are sold as of today.

```

98 • SELECT I.*
99 FROM SALE S
100 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
101 WHERE S.BILL_DATA = CURDATE();
102

```



	ITEM_ID	ITEM_NAME	PRICE
▶	103	Bag	800
	101	Pen	50
	105	Shoes	1500
	110	Headphones	1800
	109	Charger	300

10. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, Amount

```

104 • CREATE VIEW Bill_Details AS
105 SELECT
106     S.BILL_NO,
107     S.BILL_DATA,
108     S.CUST_ID,
109     S.ITEM_ID,
110     I.PRICE,
111     S.QTY_SOLD,
112     (S.QTY_SOLD * I.PRICE) AS AMOUNT
113 FROM SALE S
114 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID;
115
116 • SELECT * FROM BILL_DETAILS;
117

```



	BILL_NO	BILL_DATA	CUST_ID	ITEM_ID	PRICE	QTY_SOLD	AMOUNT
▶	1	2026-02-23	1	103	800	1	800
	2	2026-02-23	2	101	50	3	150
	3	2026-02-23	2	105	1500	2	3000
	4	2026-02-15	3	104	250	1	250
	5	2026-02-14	4	106	2200	1	2200
	6	2026-02-13	5	102	120	4	480
	7	2026-02-23	6	110	1800	1	1800
	8	2026-02-16	7	108	700	2	1400
	9	2026-02-23	5	109	300	5	1500
	10	2026-02-12	8	107	500	1	500

11. Create a view which lists the daily sales date wise for the last one week

```

119 • CREATE VIEW WEEKLY_SALES AS
120 SELECT
121     BILL_DATA,
122     SUM(S.QTY_SOLD * I.PRICE) AS TOTAL_DAILY_SALES
123 FROM SALE S
124 JOIN ITEM I ON S.ITEM_ID = I.ITEM_ID
125 WHERE BILL_DATA >= CURDATE() - INTERVAL 7 DAY
126 GROUP BY BILL_DATA;
127
128 • SELECT * FROM WEEKLY_SALES;
129

```



	BILL_DATA	TOTAL_DAILY_SALES
▶	2026-02-23	7250
	2026-02-16	1400

Experiment: 4

Question: Consider the following table: Table: CLASS (ID,Name)

```
1 • CREATE DATABASE NAKUL;  
2 • USE NAKUL;  
3 • CREATE TABLE CLASS(  
4   ID INT,  
5   NAME VARCHAR(100));  
6 • INSERT INTO  
7   CLASS(ID, NAME)  
8   VALUES  
9   (1, 'Nakul'),  
10  (2, 'Harsh'),  
11  (3, 'Ankit'),  
12  (4, 'Aryan'),  
13  (5, 'Hridyanshu'),  
14  (6, 'Ashok'),  
15  (7, 'Alok'),  
16  (8, 'Aman'),  
17  (9, 'Ashish'),  
18  (10, 'Mohit');  
19 • SELECT * FROM CLASS;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	Hridyanshu
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit

1.SET AUTOCOMMIT = 0; and START TRANSACTION;

```
22 • SET autocommit = 0;  
23 • START TRANSACTION;
```

2.INSERT INTO class VALUES (11,'Rahul');

```
24 • INSERT INTO CLASS VALUES (11, 'RAHUL');  
25 • SELECT * FROM CLASS;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	Hridyanshu
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS
	13	BRAVO
	11	RAHUL

3. UPDATE class SET name = 'Abhijeet' WHERE id= '5';

```
26 • SET SQL_SAFE_UPDATES = 0;  
27 • UPDATE CLASS SET NAME = 'ABHIJEET' WHERE ID = 5;  
28 • SELECT * FROM CLASS;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL

4. SAVEPOINT A:

```
28 • SAVEPOINT A;
```

5. INSERT INTO class VALUES (12, 'Chris');

```
30 • INSERT INTO CLASS VALUES (12, 'CHRIS');
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS

6. SAVEPOINT B;

```
31 • SAVEPOINT B;
```

7. INSERT INTO class VALUES (13, 'Bravo');

```
32 • INSERT INTO CLASS VALUES (13, 'BRAVO');  
33 • SELECT * FROM CLASS;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS
	13	BRAVO

8. SAVEPOINT C;

```
34 • SAVEPOINT C;
```

9. ROLLBACK TO B;

```
36 • ROLLBACK TO B;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL
	12	CHRIS

10. ROLLBACK TO A;

```
38 • ROLLBACK TO A;  
39 • SELECT * FROM CLASS;
```



	ID	NAME
▶	1	Nakul
	2	Harsh
	3	Ankit
	4	Aryan
	5	ABHIJEET
	6	Ashok
	7	Alok
	8	Aman
	9	Ashish
	10	Mohit
	11	RAHUL

11. COMMIT;

```
40 • COMMIT;
```

Experiment: 4.2

Consider the following table: Table: ACCOUNT_TXN

Acc_ID Holder_Name Balance

101	Aman	5000
102	Bhavna	7000
103	Chirag	6000

```
42 • CREATE TABLE ACCOUNT_TXN(  
43     ACC_ID INT,  
44     HOLDER_NAME VARCHAR(100),  
45     BALANCE INT  
46 );  
47 • INSERT INTO ACCOUNT_TXN  
48     VALUES  
49     (101, 'AMAN', 5000),  
50     (102, 'BHAVNA', 7000),  
51     (103, 'CHIRAG', 6000);  
52 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	5000
	102	BHAVNA	7000
	103	CHIRAG	6000

Perform the following TCL-based SQL script and write the output after each SELECT statement:

1. INSERT INTO ACCOUNT_TXN VALUES (104, 'Diya', 4000);

```
53 • INSERT INTO ACCOUNT_TXN  
54     VALUES  
55     (104, 'DIYA', 4000);
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	5000
	102	BHAVNA	7000
	103	CHIRAG	6000
	104	DIYA	4000

2. SAVEPOINT S1

```
56 • SET autocommit = 0;  
57 • START TRANSACTION;  
58 • SAVEPOINT S1;
```

3. UPDATE ACCOUNT_TXN SET Balance = Balance + 1500 WHERE Acc_ID = 101

```
60 • UPDATE ACCOUNT_TXN SET BALANCE = BALANCE+1500 WHERE ACC_ID = 101;  
61 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA	7000
	103	CHIRAG	6000
	104	DIYA	4000

4. UPDATE ACCOUNT_TXN SET Balance = Balance - 1000 WHERE Acc_ID = 102

```
62 • UPDATE ACCOUNT_TXN SET BALANCE = BALANCE - 1000 WHERE ACC_ID = 102;  
63 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	8000
	102	BHAVNA	6000
	103	CHIRAG	6000
	104	DIYA	4000

5. SAVEPOINT S2;

```
64 • SAVEPOINT S2;
```

6. DELETE FROM ACCOUNT_TXN WHERE Acc_ID = 103

```
65 • DELETE FROM ACCOUNT_TXN WHERE ACC_ID = 103;  
66 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	8000
	102	BHAVNA	6000
	104	DIYA	4000

7. INSERT INTO ACCOUNT_TXN VALUES (105, 'Eshan', 3000);

```
67 • INSERT INTO ACCOUNT_TXN  
68 VALUES  
69 (105, 'ESHAN', 3000);  
70 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	8000
	102	BHAVNA	6000
	104	DIYA	4000
	105	ESHAN	3000

8. SAVEPOINT S3

```
62 • SAVEPOINT S3;
```

9. UPDATE ACCOUNT_TXN SET Holder_Name = 'Bhavna Sharma' WHERE Acc_ID = 102;

```
65 • UPDATE ACCOUNT_TXN SET HOLDER_NAME = 'BHAVNA SHARMA' WHERE ACC_ID = 102;  
66 • SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	5000
	102	BHAVNA SHARMA	7000
	103	CHIRAG	6000
	104	DIYA	4000

11. ROLLBACK TO S2;

ROLLBACK TO S2;

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000

12. INSERT INTO ACCOUNT_TXN VALUES (106, 'Farah', 4500)

```
INSERT INTO ACCOUNT_TXN
VALUES
(106, 'FARAH', 4500);
SELECT * FROM ACCOUNT_TXN;
```



	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

13. SAVEPOINT S4;

SAVEPOINT S4;

14. UPDATE ACCOUNT_TXN SET Balance = Balance + 500 WHERE Acc_ID IN (101,104);

SELECT * FROM ACCOUNT_TXN

```
UPDATE ACCOUNT_TXN SET BALANCE = BALANCE + 500 WHERE ACC_ID IN (101,104);
SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	7000
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4500
	104	DIYA	4500
	106	FARAH	4500

15. ROLLBACK TO S4;

SELECT * FROM ACCOUNT_TXN;

```
ROLLBACK TO S4;
SELECT * FROM ACCOUNT_TXN;
```

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

4000

16.COMMIT

COMMIT;

17. Identify which rows are permanently saved after COMMIT.

	ACC_ID	HOLDER_NAME	BALANCE
▶	101	AMAN	6500
	102	BHAVNA SHARMA	6000
	103	CHIRAG	6000
	104	DIYA	4000
	104	DIYA	4000
	106	FARAH	4500

Assignment 5

DDL QUESTIONS (Data Definition Language)

1. Create table CLASS with appropriate data types

```
INSERT INTO CLASS VALUES  
(1, 'Nakul'),  
(2, 'Etish'),  
(4, 'Yagya');
```

OUTPUT:

	id	name
▶	1	Nakul
	2	Etish
	4	Yagya

2. Create table EMPLOYEES with specified attributes

```

CREATE TABLE EMPLOYEES (
    Employee_Id INT PRIMARY KEY,
    First_Name VARCHAR(50),
    Last_Name VARCHAR(50),
    Email VARCHAR(50),
    Phone_Number VARCHAR(20),
    Hire_Date DATE,
    Job_Id VARCHAR(20),
    Salary INT,
    Commission_Pct DECIMAL(5,2),
    Manager_Id INT,
    Department_Id INT
);

```

OUTPUT:

	Field	Type	Null	Key	Default	Extra
►	Employee_Id	int	NO	PRI	NULL	
	First_Name	varchar(50)	YES		NULL	
	Last_Name	varchar(50)	YES		NULL	
	Email	varchar(50)	YES		NULL	
	Phone_Number	varchar(20)	YES		NULL	

3. Create table Client_master with specified attributes

```

CREATE TABLE Client_master (
    ClientNo VARCHAR(10) PRIMARY KEY,
    Name VARCHAR(50),
    City VARCHAR(50),
    Bal_due INT
);

```

OUTPUT:

	Field	Type	Null	Key	Default	Extra
►	ClientNo	varchar(10)	NO	PRI	NULL	
	Name	varchar(50)	YES		NULL	
	City	varchar(50)	YES		NULL	
	Bal_due	int	YES		NULL	

4. Rename table Client_master to Client12

```
RENAME TABLE Client_master TO Client12;
```

OUTPUT:

	Tables_in_college_db
►	class
	client12
	employees

5. Display bal_due column heading as "BALANCE"

```
SELECT Bal_due AS BALANCE FROM Client12;
```

OUTPUT:

Result Grid	
	BALANCE

DML QUESTIONS (Data Manipulation Language)

Part A: Transactions and Savepoints (CLASS Table)

1. Insert + Commit

```
INSERT INTO CLASS VALUES (5, 'Rahul');
COMMIT;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Rahul
•	NULL	NULL

2. Update + Savepoint A

```
UPDATE CLASS SET name = 'Abhijeet' WHERE id = 5;  
SAVEPOINT A;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
•	NULL	NULL

3. Insert + Savepoint B

```
INSERT INTO CLASS VALUES (6, 'Chris');  
SAVEPOINT B;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris

4. Insert + Savepoint C

```
INSERT INTO CLASS VALUES (7, 'Bravo');
SAVEPOINT C;
```

OUTPUT:

	id	name
	4	Cheng
	5	Abhijeet
	6	Chris
	7	Bravo
•	NULL	NULL

5. Rollback to Savepoint B

```
ROLLBACK TO B;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
	6	Chris

6. Rollback to Savepoint A

```
ROLLBACK TO A;
```

OUTPUT:

	id	name
▶	1	Bravo
	2	Alex
	4	Cheng
	5	Abhijeet
*	HULL	HULL

PART B: Retrieving Records (EMPLOYEES Table)

1. Find out the employee id, names, and salaries of all the employees.

```
SELECT employee_id, first_name, last_name, salary  
FROM employees;
```

OUTPUT:

	employee_id	first_name	last_name	salary
▶	101	Steven	King	24000.00
	102	Neena	Kochhar	17000.00
	103	Lex	De Haan	17000.00
	104	Alexander	Hunold	9000.00
	105	Bruce	Ernst	6000.00

2. List out the employees who work under manager 100.

```
SELECT *  
FROM employees  
WHERE manager_id = 100;
```

OUTPUT:

	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

3. Find the names of the employees who have a salary greater than or equal to 4800.

```
SELECT first_name, last_name
FROM employees
WHERE salary >= 4800;
```

OUTPUT:

	first_name	last_name
▶	Steven	King
	Neena	Kochhar
	Lex	De Haan
	Alexander	Hunold
	Bruce	Ernst

4. List out the employees whose last name is 'AUSTIN'.

```
SELECT *
FROM employees
WHERE last_name = 'Austin';
```

OUTPUT:

	employee_id	first_name	last_name	email	phone_number	hire_date	job_id	salary	commission_pct	manager_id	department_id
▶	106	David	Austin	DAUSTIN	5904234569	2005-06-25	IT_PROG	4800.00	NULL	103	60
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

5. Find the names of the employees who work in departments 60, 70, and 80.

```
SELECT first_name, last_name
FROM employees
WHERE department_id IN (60,70,80);
```

OUTPUT:

	first_name	last_name
▶	Alexander	Hunold
	Bruce	Ernst
	David	Austin
	Valli	Pataballa
	Diana	Lorentz

6. Display the unique Manager_Id.

```
SELECT DISTINCT manager_id
FROM employees;
```

OUTPUT:

	manager_id
▶	NULL
	101
	102
	104
	103

Assignment 6

Updating Records (Client_master Table)

1. Insert five records into the Client_master table.

```
SELECT * FROM Client_master;
```

OUTPUT:

	ClientNO	Name	City	State	Bal_due
▶	C121	Rohit Sharma	Mumbai	Maharashtra	3000
	C122	Ananya Verma	Delhi	Delhi	6200
	C123	Karan Mehta	Pune	Maharashtra	4800
	C124	Sneha Iyer	Bangalore	Karnataka	7500
	C125	Amit Khanna	Chandigarh	Punjab	5200

2. Find the names of clients whose bal_due > 5000.

```
SELECT Name  
FROM Client_master  
WHERE Bal_due > 5000;
```

OUTPUT:

	Name
▶	Ananya Verma
	Sneha Iyer
	Amit Khanna

3. Change the bal_due of ClientNO 'C123' to Rs. 5100.

```
UPDATE Client_master  
SET Bal_due = 5100  
WHERE ClientNO = 'C123';
```

OUTPUT:

	ClientNO	Name	City	State	Bal_due
▶	C123	Karan Mehta	Pune	Maharashtra	5100
✱	NULL	NULL	NULL	NULL	NULL

ASSIGNMENT : 7

Rollback and Commit commands Create Teacher table with the following fields (Name, DeptNo, Date of joining, DeptName, Location, Salary)

```
CREATE DATABASE ASSIGNMENT6;  
USE ASSIGNMENT6;  
> CREATE TABLE Teacher (  
    Name VARCHAR(50),  
    DeptNo INT,  
    DateOfJoining DATE,  
    DeptName VARCHAR(50),  
    Location VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

(a) Insert five records

```
INSERT INTO Teacher VALUES  
( 'Nakul Arora', 1, '2020-06-15', 'Mathematics', 'Delhi', 50000),  
( 'Harsh', 2, '2019-03-10', 'Commerce', 'Mumbai', 45000),  
( 'Raj', 1, '2018-07-20', 'Mathematics', 'Delhi', 55000),  
( 'Priya', 3, '2021-01-05', 'Science', 'Chennai', 40000),  
( 'Karan', 2, '2017-09-25', 'Commerce', 'Mumbai', 48000);  
  
SELECT * FROM Teacher;
```

Name	DeptNo	DateOfJoining	DeptName	Location	Salary
Nakul Arora	1	2020-06-15	Mathematics	Delhi	50000.00
Harsh	2	2019-03-10	Commerce	Mumbai	45000.00
Raj	1	2018-07-20	Mathematics	Delhi	55000.00
Priya	3	2021-01-05	Science	Chennai	40000.00
Karan	2	2017-09-25	Commerce	Mumbai	48000.00
Karan	2	2017-09-25	Commerce	Mumbai	48000.00

(b) Give Increment of 25% salary for Mathematics Department .

```
START TRANSACTION;  
SET SQL_SAFE_UPDATES = 0;  
UPDATE Teacher  
SET Salary = Salary + (Salary * 0.25)  
WHERE DeptName = 'Mathematics';  
SELECT * FROM Teacher;
```

	Name	DeptNo	DateOfJoining	DeptName	Location	Salary
►	Nakul Arora		2020-06-15	Mathematics	Delhi	50000.00
	Harsh		2019-03-10	Commerce	Mumbai	45000.00
	Raj		2018-07-20	Mathematics	Delhi	55000.00
	Priya		2021-01-05	Science	Chennai	40000.00
	Karan		2017-09-25	Commerce	Mumbai	48000.00
	Karan		2017-09-25	Commerce	Mumbai	48000.00

(c) Perform Rollback command

ROLLBACK;

SELECT * FROM Teacher;

	Name	DeptNo	DateOfJoining	DeptName	Location	Salary
►	Nakul Arora		2020-06-15	Mathematics	Delhi	50000.00
	Harsh		2019-03-10	Commerce	Mumbai	45000.00
	Raj		2018-07-20	Mathematics	Delhi	55000.00
	Priya		2021-01-05	Science	Chennai	40000.00
	Karan		2017-09-25	Commerce	Mumbai	48000.00

(d)GiveIncrementof15%salaryforCommerce Department

UPDATE Teacher

SET Salary = Salary + (Salary * 0.15)

WHERE DeptName = 'Commerce';

SELECT * FROM Teacher;

	Name	DeptNo	DateOfJoining	DeptName	Location	Salary
►	Nakul Arora		2020-06-15	Mathematics	Delhi	50000.00
	Harsh		2019-03-10	Commerce	Mumbai	51750.00
	Raj		2018-07-20	Mathematics	Delhi	55000.00
	Priya		2021-01-05	Science	Chennai	40000.00
	Karan		2017-09-25	Commerce	Mumbai	55200.00

(e) Perform commit command

COMMIT;

ASSIGNMENT : 8

(Exercise on order by and group by clauses) Create Sales table with the following fields (Sales No, Salesname, Branch, Salesamount, DOB)

```
CREATE TABLE Sales (  
    SalesNo INT,  
    SalesName VARCHAR(50),  
    Branch VARCHAR(50),  
    SalesAmount DECIMAL(10,2),  
    DOB DATE  
);
```

(a) Insert five records

```
INSERT INTO Sales VALUES  
(1, 'Nakul Arora', 'Delhi', 10000, '1995-12-21'),  
(2, 'Harsh', 'Mumbai', 15000, '1998-05-10'),  
(3, 'Raj', 'Delhi', 20000, '1992-12-11'),  
(4, 'Priya', 'Chennai', 12000, '1997-07-25'),  
(5, 'Karan', 'Mumbai', 18000, '1994-12-05');
```

```
SELECT * FROM Sales;
```

(b) Calculate total salesamount in each branch

```
SELECT Branch, SUM(SalesAmount) AS TotalSales  
FROM Sales  
GROUP BY Branch;
```

	SalesNo	SalesName	Branch	SalesAmount	DOB
▶	1	Nakul Arora	Delhi	10000.00	1995-12-21
	2	Harsh	Mumbai	15000.00	1998-05-10
	3	Raj	Delhi	20000.00	1992-12-11
	4	Priya	Chennai	12000.00	1997-07-25
	5	Karan	Mumbai	18000.00	1994-12-05

(c) Calculate average sales amount in each branch

	Branch	TotalSales
▶	Delhi	30000.00
	Mumbai	33000.00
	Chennai	12000.00

```
SELECT Branch, AVG(SalesAmount) AS AvgSales
FROM Sales
GROUP BY Branch;
```

	Branch	AvgSales
▶	Delhi	15000.000000
	Mumbai	16500.000000
	Chennai	12000.000000

(d) Display all the salesmen, DOB who are born in the month of December as day in character format i.e. 21-Dec-09

```
SELECT SalesName, DATE_FORMAT(DOB, '%d-%b-%y') AS DOB
FROM Sales
WHERE MONTH(DOB) = 12;
```

	SalesName	DOB
▶	Nakul Arora	21-Dec-95
	Raj	11-Dec-92
	Karan	05-Dec-94

(e) Display the name and DOB of salesman in alphabetical order of the month.

```
SELECT SalesName, DOB
FROM Sales
ORDER BY DATE_FORMAT(DOB, '%M');
```

	SalesName	DOB
▶	Nakul Arora	1995-12-21
	Raj	1992-12-11
	Karan	1994-12-05
	Priya	1997-07-25
	Harsh	1998-05-10

ASSIGNMENT : 9

Consider the following tables namely “DEPARTMENTS” and “EMPLOYEES” Their schemas are as follows,

Departments (dept_no , dept_name , dept_location); Employees (emp_id , emp_name , emp_salary,dept_no);

```
CREATE TABLE Departments (  
    dept_no INT,  
    dept_name VARCHAR(50),  
    dept_location VARCHAR(50)  
);
```

```
CREATE TABLE Employees (  
    emp_id INT,  
    emp_name VARCHAR(50),  
    emp_salary DECIMAL(10,2),  
    dept_no INT  
);
```

a) Develop a query to grant all privileges of employees table into departments table

```
GRANT ALL PRIVILEGES ON Employees TO 'root'@'localhost';
```

b) Develop a query to grant some privileges of employees table into departments table

```
GRANT SELECT, INSERT ON Employees TO 'root'@'localhost';
```

c) Develop a query to revoke all privileges of employees table from departments table

```
CREATE USER 'student'@'localhost' IDENTIFIED BY '1234';  
GRANT ALL PRIVILEGES ON Employees TO 'student'@'localhost';  
REVOKE ALL PRIVILEGES ON Employees FROM 'student'@'localhost';
```

d) Develop a query to revoke some privileges of employees table from departments table

```
REVOKE SELECT, INSERT ON Employees FROM 'root'@'localhost';
```

e) Write a query to implement the save point.

```
START TRANSACTION;  
SAVEPOINT sp1;  
UPDATE Employees  
SET emp_salary = emp_salary + 1000;  
SELECT * FROM Employees;
```

	emp_id	emp_name	emp_salary	dept_no
▶	101	Nakul Arora	51000.00	1
	102	Harsh	61000.00	2
	103	Raj	56000.00	3
	104	Priya	46000.00	2
	105	Karan	71000.00	3
	106	Sneha	41000.00	4
	107	Vikas	66000.00	5

ROLLBACK TO sp1;

SELECT * FROM Employees;

	emp_id	emp_name	emp_salary	dept_no
--	--------	----------	------------	---------

COMMIT;

ASSIGNMENT : 10

Using the tables "DEPARTMENTS" and "EMPLOYEES" perform the following queries

```
CREATE DATABASE ASS10;
USE ASS10;
CREATE TABLE Departments (
    dept_no INT,
    dept_name VARCHAR(50),
    dept_location VARCHAR(50)
);
INSERT INTO Departments VALUES
(1, 'HR', 'Delhi'),
(2, 'Finance', 'Mumbai'),
(3, 'IT', 'Bangalore'),
(4, 'Marketing', 'Chennai'),
(5, 'Sales', 'Hyderabad');

CREATE TABLE Employees (
    emp_id INT,
    emp_name VARCHAR(50),
    emp_salary DECIMAL(10,0;2),
    dept_no INT
);
INSERT INTO Employees VALUES
(101, 'Nakul Arora', 50000, 1,
(102, 'Harsh', 60000, 2,
(103, 'Raj', 55000, 3),
(104, 'Priya', 45000, 2,
(105, 'Karan', 70000, 3,
(106, 'Sneha', 41000, 4,
(107, 'Vikas', 65000, 5,
```

a) Display the employee details, departments that the departments are same in both the emp and dept.

```
SELECT *
FROM Employees e
INNER JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_id	emp_name	emp_salary	dept_no	dept_name	dept_locati
►	101	Nakul Arora	50000.00	1	HR	Delhi
	102	Harsh	60000.00	2	Finance	Mumbai
	103	Raj	55000.00	3	IT	Bangalore
	104	Priya	45000.00	2	Finance	Mumbai
	105	Karan	70000.00	3	IT	Bangalore
	106	Sneha	40000.00	4	Marketing	Chennai
	107	Vikas	65000.00	5	Sales	Hyderabad

b) Display the employee name and Department name by implementing a left outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
LEFT JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_name	dept_name
►	Nakul Arora	HR
	Harsh	Finance
	Raj	IT
	Priya	Finance
	Karan	IT
	Sneha	Marketing
	Vikas	Sales

c) Display the employee name and Department name by implementing a right outer join.

```
SELECT e.emp_name, d.dept_name
FROM Employees e
RIGHT JOIN Departments d
ON e.dept_no = d.dept_no;
```

	emp_name	dept_name
►	Nakul Arora	HR
	Priya	Finance
	Harsh	Finance
	Karan	IT
	Raj	IT
	Sneha	Marketing
	Vikas	Sales

d) Display the details of those who draw the salary greater than the average salary

```
SELECT *  
FROM Employees  
WHERE emp_salary > (SELECT AVG(emp_salary) FROM Employees);
```

	emp_id	emp_name	emp_salary	dept_no
▶	102	Harsh	60000.00	2
	105	Karan	70000.00	3
	107	Vikas	65000.00	5